

ACTIVIDAD 1

Estructura de datos

Docente: Alberto Lemus Mera



ÍNDICE

Docente: Alberto Lemus Mera.....	1
Objetivo.....	3
Instrucciones.....	3
Código.....	3
App.java.....	3
Node.java.....	6
tiposDeDatos.java.....	6
Contactos.java.....	7
ListaEnlazada.java.....	8
Corrida del código.....	11

Objetivo

Aplicar los conocimientos de estructuras de datos dinámicas, tipos de datos y programación orientada a objetos para implementar y manipular listas enlazadas en Java, mejorando la comprensión de la gestión de memoria, las referencias entre objetos y la manipulación de diversos tipos de datos.

Instrucciones

1. Desarrolla un programa en Java que implemente diferentes tipos de listas enlazadas (simplemente enlazadas, doblemente enlazadas y circulares) y asegúrate de que realice operaciones básicas sobre estas estructuras.
2. El programa permitirá al usuario seleccionar el tipo de lista, insertar elementos de diferentes tipos de datos, eliminar elementos, buscar elementos y mostrar los elementos de la lista.
3. Crea cuatro clases: Node, LinkedList, DataTypeExamples, y Main.
 - Node representará un nodo en la lista.
 - LinkedList gestionará las operaciones en la lista.
 - DataTypeExamples incluirá métodos para demostrar el uso de diferentes tipos de datos.
 - Main será el punto de entrada del programa para interactuar con el usuario.
4. Implementa métodos en la clase LinkedList para manejar nodos de diferentes tipos de datos:
 - Ejemplos de tipos de datos: En la clase DataTypeExamples, implementa métodos que utilicen tipos de datos primitivos, complejos y abstractos, mostrando su aplicación práctica en contexto con las listas.
5. Utiliza la clase Main para crear un menú interactivo donde los usuarios puedan elegir el tipo de lista, realizar operaciones, interactuar con ejemplos de tipos de datos y ver los resultados en tiempo real.
6. Implementa un sistema sencillo de gestión de contactos utilizando listas enlazadas, donde cada contacto es un objeto que incluye información como nombre, dirección y número de teléfono (ejemplo de tipo de dato complejo y abstracto).

Código

App.java

```
import java.util.Scanner;

public class App {
    public static void main(String[] args) throws Exception {
        Scanner scanner = new Scanner(System.in);
        tiposDeDatos ejemplos = new tiposDeDatos();
        while (true) {
            System.out.println("\n-- Menu --");
            System.out.println("1. Ejemplos de tipos de datos y listas");
```

```
System.out.println("2. Sistema de gestion de contactos");
System.out.println("3. Salir");
int opmenu = scanner.nextInt();
scanner.nextLine();
if (opmenu == 1) {
    ejemplos.demostrarTipos();
} else if (opmenu == 2) {
    menuContactos(scanner);
} else if (opmenu == 3) {
    System.out.println("Saliendo...");
    break;
} else {
    System.out.println("Opcion no valida");
}
}
scanner.close();
}

public static void menuContactos (Scanner scanner) {
    System.out.println("\n¿Qué tipo de lista deseas usar para los
contactos?");
    System.out.println("1. Simple");
    System.out.println("2. Doble");
    System.out.println("3. Circular");
    System.out.print("Opcion: ");
    int tipoLista = scanner.nextInt();
    scanner.nextLine();
    ListaEnlazada<Contactos> listaContactos = new
ListaEnlazada<>(tipoLista);
    int op = 0;
    do {
        System.out.println("\n-- Gestion de Contactos --");
        System.out.println("1. Agregar contacto");
        System.out.println("2. Mostrar contactos");
        System.out.println("3. Eliminar un contacto");
        System.out.println("4. Buscar un contacto");
        System.out.println("5. Regresar al menu principal");
        System.out.print("Opcion: ");
        op = scanner.nextInt();
        scanner.nextLine();
    }
```

```
switch (op) {
    case 1:
        System.out.print("Nombre: ");
        String nombre = scanner.nextLine();
        System.out.print("Direccion: ");
        String direccion = scanner.nextLine();
        System.out.print("Telefono: ");
        String telefono = scanner.nextLine();
        listaContactos.appendNode(new Contactos(nombre, direccion,
telefono));
        System.out.println("Contacto agregado");
        break;
    case 2:
        listaContactos.showlist();
        break;
    case 3:
        if (listaContactos.isEmpty()) {
            System.out.println("La lista esta vacia");
            break;
        }
        listaContactos.showlist();
        System.out.print("Escribe el nombre del contacto a borrar: ");
        String nombreBorrar = scanner.nextLine();
        listaContactos.delete(new Contactos(nombreBorrar, "", ""));
        break;
    case 4:
        System.out.print("Escribe el nombre del contacto a borrar: ");
        String nombreBuscar = scanner.nextLine();
        boolean existe = listaContactos.search(new
Contactos(nombreBuscar, "", ""));
        if (existe) {
            System.out.println("Contacto encontrado");
        } else {
            System.out.println("Contacto no encontrado");
        }
        break;
    case 5:
        System.out.println("Saliendo...");
        break;
    default:
        System.out.println("Opcion no valida");
}
```

```
        break;
    }
} while (op != 5);
}
```

Node.java

```
public class Node<T> {
    T data;
    Node<T> next;
    Node<T> prev;

    public Node(T data) {
        this.data = data;
        this.next = null;
        this.prev = null;
    }

    public String toString() {
        return data != null ? data.toString() : "null";
    }
}
```

tiposDeDatos.java

```
public class tiposDeDatos {
    public void demostrarTipos() {
        System.out.println("\nDemostracion de diferentes tipos de datos y listas");
        // 1. Tipos de datos primitivos
        ListaEnlazada<Integer> listaPrimitivos = new ListaEnlazada<>(1);
        listaPrimitivos.appendNode(1);
        listaPrimitivos.appendNode(2);
        listaPrimitivos.appendNode(3);
        System.out.print("Lista de numeros enteros: ");
        listaPrimitivos.showlist();
        // 2. Tipos de datos complejos
        ListaEnlazada<String> listaComplejos = new ListaEnlazada<>(2);
        listaComplejos.appendNode("Hola");
        listaComplejos.appendNode("Buenas tardes");
    }
}
```

```
listaComplejos.appendNode("Adios");
System.out.print("Lista de palabras: ");
listaComplejos.showlist();
// 3. Tipos abstractos
ListaEnlazada<Contactos> listaAbstractos = new ListaEnlazada<>(3);
listaAbstractos.appendNode(new Contactos("Erick", "Felipe Sevilla 213",
"4424635378"));
listaAbstractos.appendNode(new Contactos("Vale", "Palermo 107",
"4424777457"));
listaAbstractos.appendNode(new Contactos("Daisy", "1 de Mayo",
"4424644984"));
System.out.print("Lista datos abstractos: ");
listaAbstractos.showlist();
}
}
```

Contactos.java

```
public class Contactos {
    public String nombre;
    public String direccion;
    public String telefono;

    public Contactos(String nombre, String direccion, String telefono) {
        this.nombre = nombre;
        this.direccion = direccion;
        this.telefono = telefono;
    }

    public String getNombre() {
        return nombre;
    }
    public String getDireccion() {
        return direccion;
    }
    public String getTelefono() {
        return telefono;
    }

    @Override
    public String toString() {
```

```
        return "\nNombre del contacto: " + nombre + ", Direccion: " + direccion
+ ", telefono: " + telefono;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Contactos contacto = (Contactos) obj;
        return nombre.trim().equalsIgnoreCase(contacto.nombre.trim());
    }
}
```

ListaEnlazada.java

```
public class ListaEnlazada<T> {
    public Node<T> head;
    public Node<T> tail; // Listas circulares
    public int tipoLista; // 1. Simples 2. Doble 3. Circulares
    public int size;

    // Constructor
    public ListaEnlazada(int tipoLista) {
        this.tipoLista = tipoLista;
        this.head = null;
        this.tail = null;
        this.size = 0;
    }

    // Metodos
    // 1. Saber si la lista esta vacia
    public boolean isEmpty() {
        return head == null;
    }

    // 2. Insercion de nodos(datos) al final de la lista
    public void appendNode(T data) {
        Node<T> nuevo = new Node<>(data);
        // Verficar si la lista esta vacia
        if (isEmpty()) {
            head = nuevo;
        }
    }
}
```

```
tail = nuevo;
// Unicamente para listas circulares
if (tipoLista == 3) {
    head.next = head;
}
} else {
    if (tipoLista == 1 ) {
        Node<T> actual = head;
        while (actual.next != null) {
            actual = actual.next;
        }
        actual.next = nuevo;
        tail = nuevo;
    } else if (tipoLista == 2) {
        tail.next = nuevo;
        nuevo.prev = tail;
        tail = nuevo;
    } else if (tipoLista == 3) {
        tail.next = nuevo;
        tail = nuevo;
        nuevo.next = head;
    }
}
size++;
}

// 3. Mostrar los elementos de las listas
public void showlist() {
    // Verificar si la lista esta vacia
    if (isEmpty()) {
        System.out.println("La lista esta vacia");
        return;
    }
    System.out.print("\nElementos de la lista: ");
    Node <T> actual = head;
    if (tipoLista == 2) {
        System.out.print("null <- ");
    }
    do {
        System.out.print(actual.data + " -> ");
        // 1 -> 2 -> 3
    } while (actual.next != null);
}
```

```
        actual = actual.next;
        if (tipoLista == 3 && actual == head) {
            break;
        }
    } while (actual != null);
    if (tipoLista == 3) {
        System.out.println("Fin de la lista circular");
    } else {
        System.out.println("null");
    }
}

// 4. Buscar elementos
public boolean search(T data) {
    // Verificar si la lista esta vacia
    if (isEmpty()) return false;
    Node <T> actual = head;
    do {
        if (actual.data.equals(data)) {
            return true;
        }
        actual = actual.next;
        if (tipoLista == 3 && actual == head) break;
    } while (actual != null);
    return false;
}

// 5. Eliminar algun dato
public void delete(T data) {
    if (isEmpty()) {
        System.out.println("La lista esta vacia");
        return;
    }
    Node<T> actual = head;
    Node<T> anterior = null;
    boolean encontrado = false;
    // i
    do {
        if (actual.data.equals(data)) {
            encontrado = true;
            break;
        }
    } while (actual != null);
    if (encontrado) {
        if (anterior == null) {
            head = head.next;
        } else {
            anterior.next = actual.next;
        }
    }
}
```

```
    }  
    anterior = actual;  
    actual = actual.next;  
    if (tipoLista == 3 && actual == head) break;  
} while (actual != null);  
if (!encontrado) {  
    System.out.println("Elemento no encontrado");  
    return;  
}  
// Caso cuando solo hay un elemento  
if (size == 1) {  
    head = null;  
    tail = null;  
} else if (actual == head) { // Eliminar el primer elemento  
    head = head.next;  
    if (tipoLista == 2) head.prev = null;  
    if (tipoLista == 3) tail.next = head;  
} else if (actual == tail) { // Eliminar el ultimo elemmento  
    tail = anterior;  
    if (tipoLista == 1 || tipoLista == 2) tail.next = null;  
    if (tipoLista == 3) tail.next = head;  
} else { // Eliminar algun dato de en medio de la lista  
    anterior.next = actual.next;  
    if (tipoLista == 2 && actual.next != null) {  
        actual.next.prev = anterior;  
    }  
}  
size--;  
System.out.println("Elemento eliminado");  
}
```

Corrida del código

```
TECMI/Estructura de datos/Tarea1 via v2S.0.3 took 59m32s  
> java App.java
```

```
-- Menu --  
1. Ejemplos de tipos de datos y listas  
2. Sistema de gestion de contactos  
3. Salir  
1  
  
Demostracion de diferentes tipos de datos y listas  
Lista de numeros enteros:  
Elementos de la lista: 1 → 2 → 3 → null  
Lista de palabras:  
Elementos de la lista: null ← Hola → Buenas tardes → Adios → null  
Lista datos abstractos:  
Elementos de la lista:  
Nombre del contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 →  
Nombre del contacto: Vale, Direccion: Palermo 107, telefono: 4424777457 →  
Nombre del contacto: Daisy, Direccion: 1 de Mayo, telefono: 4424644984 → Fin de la lista circular
```

```
-- Menu --  
1. Ejemplos de tipos de datos y listas  
2. Sistema de gestion de contactos  
3. Salir  
█
```

```
-- Menu --  
1. Ejemplos de tipos de datos y listas  
2. Sistema de gestion de contactos  
3. Salir  
2  
  
¿Qué tipo de lista deseas usar para los contactos?  
1. Simple  
2. Doble  
3. Circular  
Opcion: 1
```

```
-- Gestion de Contactos --  
1. Agregar contacto  
2. Mostrar contactos  
3. Eliminar un contacto  
4. Buscar un contacto  
5. Regresar al menu principal  
Opcion: 1  
Nombre: Erick  
Direccion: Felipe Sevilla  
Telefono: 4424635378  
Contacto agregado
```

```
-- Gestion de Contactos --  
1. Agregar contacto  
2. Mostrar contactos  
3. Eliminar un contacto  
4. Buscar un contacto  
5. Regresar al menu principal  
Opcion: 1  
Nombre: Daisy  
Direccion: 1ro De Mayo  
Telefono: 44246683218  
Contacto agregado
```

```
-- Gestion de Contactos --  
1. Agregar contacto  
2. Mostrar contactos  
3. Eliminar un contacto  
4. Buscar un contacto  
5. Regresar al menu principal  
Opcion:
```

```
-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista:
Nombre del contacto: Erick, Direccion: Felipe Sevilla, telefono: 4424635378 →
Nombre del contacto: Daisy, Direccion: 1ro De Mayo, telefono: 44426683218 → null

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

Elementos de la lista:
Nombre del contacto: Erick, Direccion: Felipe Sevilla, telefono: 4424635378 →
Nombre del contacto: Daisy, Direccion: 1ro De Mayo, telefono: 44426683218 → null
Escribe el nombre del contacto a borrar: Erick
Elemento eliminado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista:
Nombre del contacto: Daisy, Direccion: 1ro De Mayo, telefono: 44426683218 → null

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: █
```

```
-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista:
Nombre del contacto: Daisy, Direccion: 1ro De Mayo, telefono: 44426683218 →
Nombre del contacto: , Direccion: , telefono: →
Nombre del contacto: Valeria, Direccion: Palermo 107, telefono: 4425189756 → null

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

Elementos de la lista:
Nombre del contacto: Daisy, Direccion: 1ro De Mayo, telefono: 44426683218 →
Nombre del contacto: , Direccion: , telefono: →
Nombre del contacto: Valeria, Direccion: Palermo 107, telefono: 4425189756 → null
Escribe el nombre del contacto a borrar:
Elemento eliminado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 4
Escribe el nombre del contacto a borrar: Valeria
Contacto encontrado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: █
```

```

TECMI/Estructura de datos/Tarea1 via v25.0.3 took 11m19s
} java App.java

-- Menu --
1. Ejemplos de tipos de datos y listas
2. Sistema de gestion de contactos
3. Salir
2

¿Qué tipo de lista deseas usar para los contactos?
1. Simple
2. Doble
3. Circular
Opcion: 2

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 1
Nombre: Erick
Direccion: Felipe Sevilla 213
Telefono: 4424635378
Contacto agregado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista:
null ← Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → null

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: █

2. Doble
3. Circular
Opcion: 3

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 1
Nombre: Erick
Direccion: Felipe Sevilla 213
Telefono: 4424635378
Contacto agregado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 1
Nombre: Vale
Direccion: Palermo 107
Telefono: 442469831
Contacto agregado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista: Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Contacto: Vale, Direccion: Palermo 107, telefono: 442469831 → Fin de la lista circular

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion:

```

```
Opcion: 1
Nombre: Vale
Direccion: Palermo 107
Telefono: 4424569831
Contacto agregado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista: Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Contacto: Vale, Direccion: Palermo 107, telefono: 4424569831 → Fin de la lista circular

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

Elementos de la lista: Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Contacto: Vale, Direccion: Palermo 107, telefono: 4424569831 → Fin de la lista circular
Escribe el nombre del contacto a borrar: vale
Elemento eliminado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista: Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Fin de la lista circular

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 1
Nombre: Vale
Direccion: Palermo 107
Telefono: 4426531563
Contacto agregado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 2

Elementos de la lista:
null ← Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Contacto: Vale, Direccion: Palermo 107, telefono: 4426531563 → null

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

Elementos de la lista:
null ← Contacto: Erick, Direccion: Felipe Sevilla 213, telefono: 4424635378 → Contacto: Vale, Direccion: Palermo 107, telefono: 4426531563 → null
Escribe el nombre del contacto a borrar: erick
Elemento eliminado

-- Gestion de Contactos --
1. Agregar contacto
2. Mostrar contactos
3. Eliminar un contacto
4. Buscar un contacto
5. Regresar al menu principal
Opcion: 3

Elementos de la lista:
null ← Contacto: Vale, Direccion: Palermo 107, telefono: 4426531563 → null
Escribe el nombre del contacto a borrar: 
```